№10 ООП

План

- принципы SOLID
  - Принцип единственной ответственности
    - Пример
  - принцип открытости-закрытости
  - Принцип подстановки Лискоу
    - Пример нарушения принципа
  - Принцип разделения интерфейсов
  - Принцип инверсии зависимости

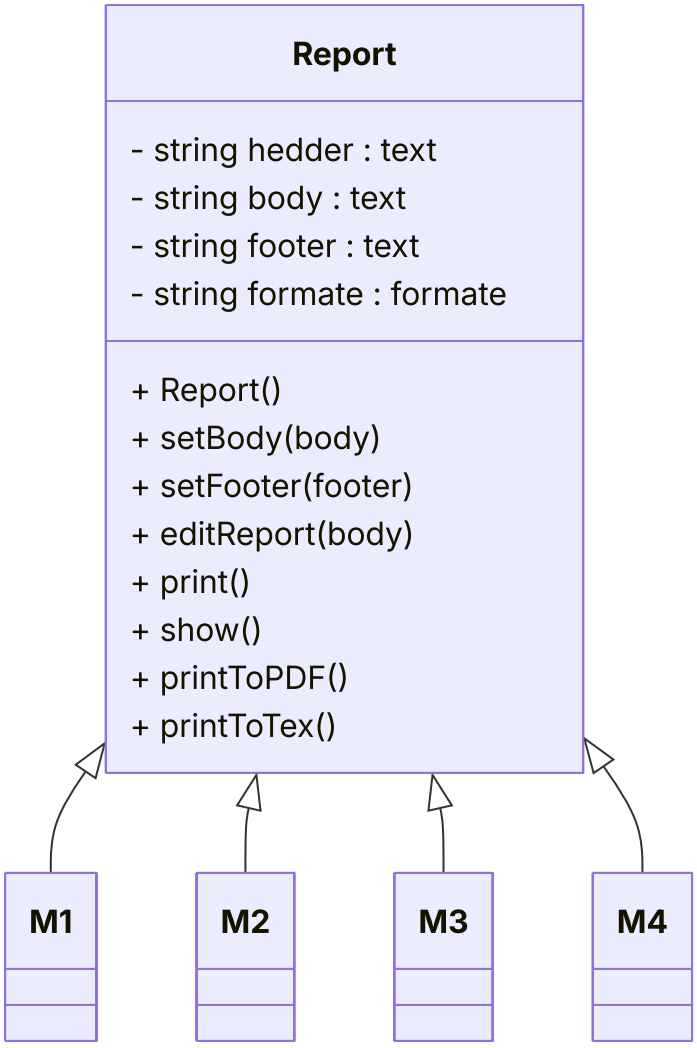
принципы SOLID

Принцип единственной ответственности

S - SRP. *Принцип единственности ответственности*. Каждый класс или функция или модуль должна отвечать только за одну вещь.

> У каждого класса должна быть *только одна причина* для его изменения.
>
> Буханов Д. Г.

Пример



Допустим у нас есть класс `Report` и на основе него построено 4 модуля. Модули `M1` и `M2` используют только *сеттеры*, а `M3` `M4` только *методы вывода*. В случае *изменений* в классе *в части сеттеров* нам придётся *перекомпилировать все модули*, даже те, которые *не используют сеттеры*. Это проблема.

Нам следует *разделить* `Report` *на два класса*, например `Report` и `Printer`.

С другой стороны *чрезмерное следование* тоже *может создавать проблемы*, так как создаёт много маленьких сущностей.

принцип открытости-закрытости

O - OCP. *Принцип открытости-закрытости*. Это принцип при котором программные сущности должны быть открыты для расширения и закрыты для модификации.

> Классы должны быть открыты для расширения, но классы должны быть закрыты от модификации.
>
> Буханов Д. Г.

Существует *две трактовки*:

- *Принцип Мейера 1988 года* - наследники могут либо переопределить метод, либо оставить базовую реализацию.
- *Полиморфный принцип* - родительский класс должен быть чистым интерфейсом.

Принцип подстановки Лискоу

L - LSP. *Принцип подстановки Лискоу*. Принцип в котором в клиентском коде нужно использовать классы наследники вместо базового класса.

> Если для некоторого типа объектов x существуют свойства $q(x_i)$, то для всех объектов класса y должно существовать $q(y_i)$, при условии, что y является наследником x.
>
> Буханов Д. Г.

Пример нарушения принципа

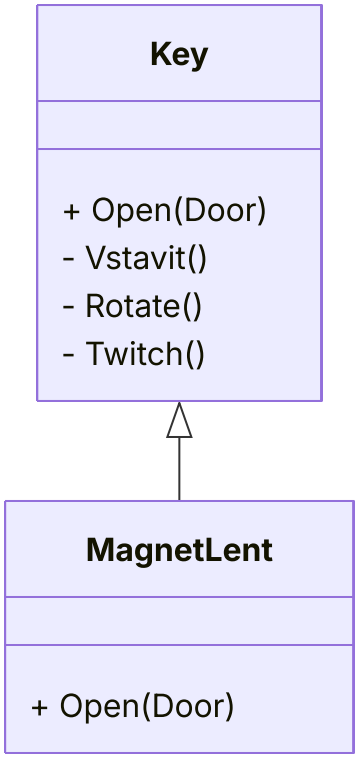
```
class Rectangle {
     int h;
     int w;
public:
     Rectangle() : h(), w() {};
     virtual void setHeight(int newH) {h = newH};
     virtual void setWeidth(int newW) {w = newW};
     virtual int area() {return h * w};
}

class Square : public Rectangle {
     int side;
public:
```

```
    void setHeight(int newH) {side = newH};
    void setWeidth(int newW) {side = newW};
    int area() {return side * side};
}

Rectangle processing_area(Rectangle &r) { // Ошибка тут
потому что интерфейс не предусматривает возможность ошибки
разных длинн сторон квадрата. Для исправления нужно создать
третий родительский класс Shape в котором будет поле area.
    r.setHeight(4);
    r.setWeidth(5);
    return r;
}

Rectangle r();
Square s();

r = processing_area(r);
s = processing_area(s);

r.area(); // 20
s.area(); // 25
```

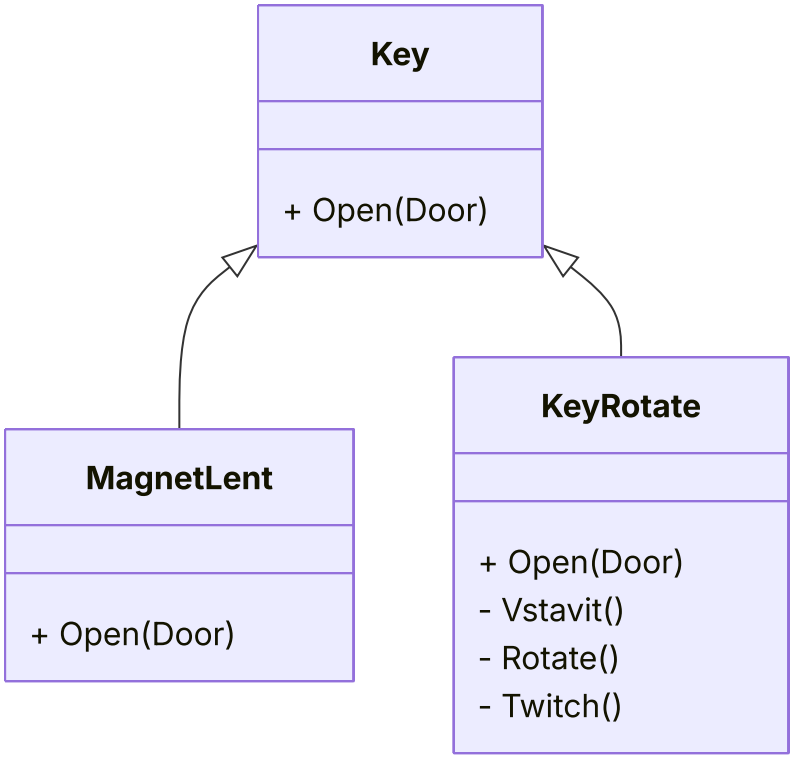
Принцип разделения интерфейсов

I - ISP. *Принцип разделения интерфейсов*. На каждую абстрактную штуку нужен отдельный интерфейс.

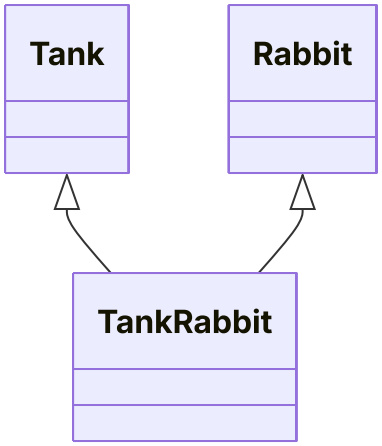
> Лучше много маленьких интерфейсов, чем один большой.
>
> Буханов Д. Г.



Такое наследование *не предусматривает дополнительные возможности*, которые могут иметь последующие классы, *так как определяет конкретный класс родительским*. Следует учредить *класс общего ключа*.



Так же можно использовать множественное наследование из родительских классов, что бы добавить больше гибкости.



Принцип инверсии зависимости

D - DIP. *Принцип инверсии зависимости*. Абстрактные классы не должны зависеть от детальных.

> Зависимость от абстракции, а не от конкретного класса, конкретный класс зависит от интерфейса.

```
class Sparrow {
public:
    void eat(int f) {};
}

class BirdReeder {
public:
    void feed(Sparrow) { // Жесткая зависимость от
воробья, что плохо.
        s→eat(f);
    }
}

BerdFeeder bd;
bd.feed(bird);
```

Что бы это исправить, нужно *сделать кормушку абстрактным классом*, и уже от него *наследовать кормушку для воробьёв*.

То же самое лучше сделать для птиц, создать абстрактный класс `Bird`.